

```
# Statement:Use the Google stock prices dataset and design a time series
# analysis and prediction system using RNN.
# To study and understand Recurrent Neural Network by doing analysis and
# designing prediction systems.
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from keras.models import Sequential
from keras.layers import Dense,LSTM,Dropout
```

```
data = pd.read_csv('C:/Users/DELL-PC/DL/GOOGL.csv')
data.head()
```

	Date	Open	High	Low	Close	Adj Close
Volume						
0	2004-08-19	50.050049	52.082081	48.028027	50.220219	50.220219
44659096						
1	2004-08-20	50.555557	54.594597	50.300301	54.209209	54.209209
22834343						
2	2004-08-23	55.430431	56.796799	54.579578	54.754753	54.754753
18256126						
3	2004-08-24	55.675674	55.855858	51.836838	52.487488	52.487488
15247337						
4	2004-08-25	52.532532	54.054054	51.991993	53.053055	53.053055
9188602						

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4431 entries, 0 to 4430
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        4431 non-null   object
1   Open        4431 non-null   float64
2   High        4431 non-null   float64
3   Low         4431 non-null   float64
4   Close       4431 non-null   float64
5   Adj Close   4431 non-null   float64
6   Volume      4431 non-null   int64
dtypes: float64(5), int64(1), object(1)
memory usage: 242.4+ KB
```

```
data["Close"]=pd.to_numeric(data.Close,errors='coerce')
data = data.dropna()
trainData = data.iloc[:,4:5].values
```

```
data.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4431 entries, 0 to 4430
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   Date        4431 non-null   object
 1   Open        4431 non-null   float64
 2   High        4431 non-null   float64
 3   Low         4431 non-null   float64
 4   Close       4431 non-null   float64
 5   Adj Close   4431 non-null   float64
 6   Volume      4431 non-null   int64
dtypes: float64(5), int64(1), object(1)
memory usage: 242.4+ KB

sc = MinMaxScaler(feature_range=(0,1))
trainData = sc.fit_transform(trainData)
trainData.shape

(4431, 1)

X_train = []
y_train = []

for i in range(60, 1149): # 60 = timestep
    X_train.append(trainData[i-60:i, 0])
    y_train.append(trainData[i, 0])

X_train, y_train = np.array(X_train), np.array(y_train)

X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1)) #adding
the batch_size
X_train.shape

(1089, 60, 1)

model = Sequential()
model.add(LSTM(units=100, return_sequences = True, input_shape
=(X_train.shape[1],1)))
model.add(Dropout(0.2))
model.add(LSTM(units=100, return_sequences = True))
model.add(Dropout(0.2))
model.add(LSTM(units=100, return_sequences = True))
model.add(Dropout(0.2))
model.add(LSTM(units=100, return_sequences = False))
model.add(Dropout(0.2))
model.add(Dense(units =1))
model.compile(optimizer='adam', loss="mean_squared_error")

C:\Users\DELL-PC\anaconda3\Lib\site-packages\keras\src\layers\rnn\rnn.py:199:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer.

```

When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

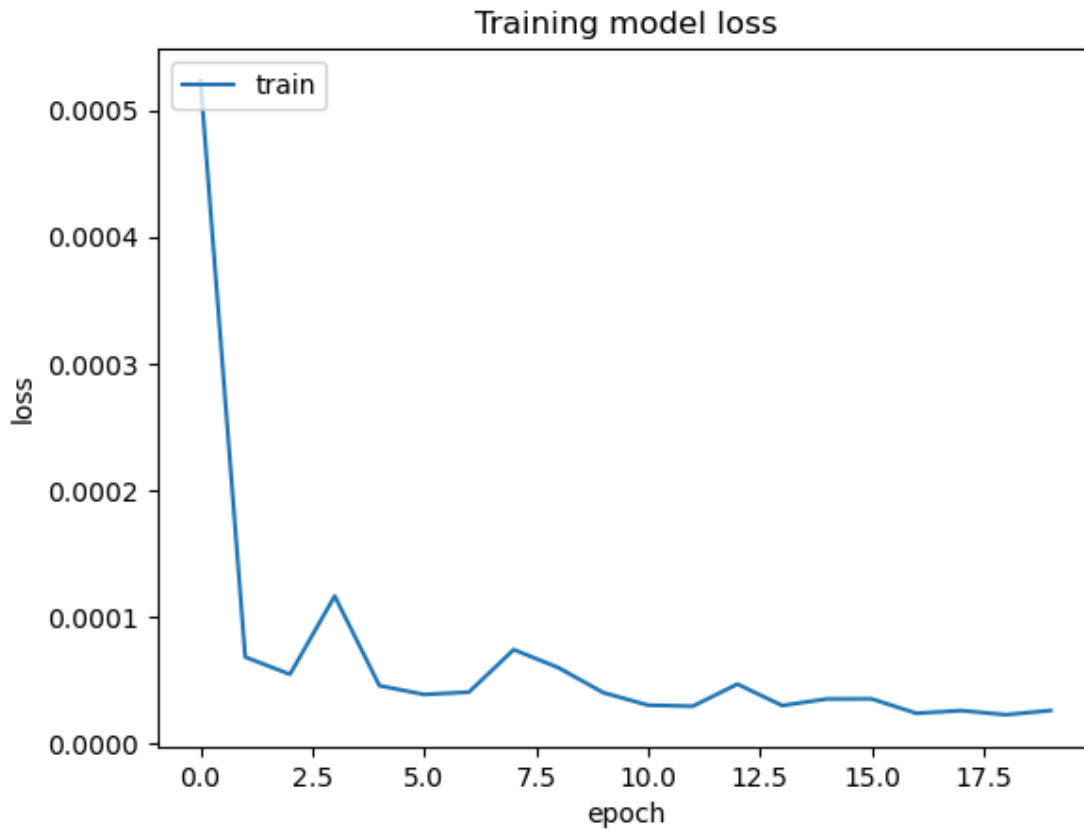
```
hist = model.fit(X_train, y_train, epochs = 20, batch_size = 32, verbose=2)
```

```
Epoch 1/20
35/35 - 19s - 545ms/step - loss: 5.2333e-04
Epoch 2/20
35/35 - 11s - 312ms/step - loss: 6.8588e-05
Epoch 3/20
35/35 - 7s - 205ms/step - loss: 5.4983e-05
Epoch 4/20
35/35 - 8s - 222ms/step - loss: 1.1667e-04
Epoch 5/20
35/35 - 7s - 198ms/step - loss: 4.5973e-05
Epoch 6/20
35/35 - 7s - 213ms/step - loss: 3.9052e-05
Epoch 7/20
35/35 - 10s - 273ms/step - loss: 4.0916e-05
Epoch 8/20
35/35 - 7s - 204ms/step - loss: 7.4439e-05
Epoch 9/20
35/35 - 10s - 290ms/step - loss: 6.0198e-05
Epoch 10/20
35/35 - 10s - 290ms/step - loss: 4.0522e-05
Epoch 11/20
35/35 - 7s - 199ms/step - loss: 3.0642e-05
Epoch 12/20
35/35 - 10s - 288ms/step - loss: 2.9768e-05
Epoch 13/20
35/35 - 11s - 304ms/step - loss: 4.7231e-05
Epoch 14/20
35/35 - 8s - 227ms/step - loss: 3.0327e-05
Epoch 15/20
35/35 - 8s - 222ms/step - loss: 3.5524e-05
Epoch 16/20
35/35 - 7s - 212ms/step - loss: 3.5577e-05
Epoch 17/20
35/35 - 8s - 214ms/step - loss: 2.4217e-05
Epoch 18/20
35/35 - 8s - 221ms/step - loss: 2.6354e-05
Epoch 19/20
35/35 - 8s - 220ms/step - loss: 2.3085e-05
Epoch 20/20
35/35 - 10s - 290ms/step - loss: 2.6444e-05
```

```

plt.plot(hist.history['loss'])
plt.title('Training model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train'], loc='upper left')
plt.show()

```



```

data["Close"] = pd.to_numeric(data["Close"], errors='coerce')

```

```

# Drop NaN values
data = data.dropna()

```

```

# Select Close column
data = data.iloc[:, 4:5]

```

```

# Create y_test
y_test = data.iloc[60:, 0:].values

```

```

# Prepare input for model
inputClosing = data.iloc[:, 0:].values
inputClosing_scaled = sc.transform(inputClosing)

```

```

# Create X_test
X_test = []

```

```

timestep = 60
length = len(data)

for i in range(timestep, length):
    X_test.append(inputClosing_scaled[i - timestep:i, 0])

X_test = np.array(X_test)

# Reshape for LSTM (3D)
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

print(X_test.shape)

(4371, 60, 1)

y_pred = model.predict(X_test)
y_pred

137/137 ----- 13s 89ms/step

array([[0.0125452 ],
       [0.01205804],
       [0.01157946],
       ...,
       [0.7010528 ],
       [0.70383924],
       [0.70787394]], shape=(4371, 1), dtype=float32)

predicted_price = sc.inverse_transform(y_pred)

plt.plot(y_test, color = 'red', label = 'Actual Stock Price')
plt.plot(predicted_price, color = 'green', label = 'Predicted Stock Price')
plt.title('Google stock price prediction')
plt.xlabel('Time')
plt.ylabel('Stock Price')
plt.legend()
plt.show()

```

